You asked:

You said: I want to build a smart cooking conversational and adaptive assistant using local models, a backend running on a local server and a front-end on a tablet app

I want to build a smart cooking conversational and adaptive assistant using local models, a backend running on a local server and a front-end on a tablet app

Claude Replied:

Claude responded: This fits naturally with the split architecture — HA stays the backend for inventory and device control, and this local stack becomes the actual cooking compan…

Thought for 47s
Thought for 47s

This fits naturally with the split architecture — HA stays the backend for inventory and device control, and this local stack becomes the actual cooking companion layer.

Voice pipeline (three stages, since there's no solid production-ready local speech-to-speech model yet):

- **STT:** faster-whisper or whisper.cpp — whisper-small/medium is accurate enough for kitchen use and fast on a decent GPU
- **LLM:** Ollama running Qwen2.5 (7B–14B) or Llama 3.1 8B — both handle recipe-step reasoning and tool-calling well
- **TTS:** Piper (very low latency, runs fine on CPU) or Coqui XTTS if you want a more natural voice and can spend the latency

Worth flagging: chaining STT→LLM→TTS means real latency compared to something like GPT-4o Advanced Voice's native speech-to-speech — think 1.5–3s round trips, not instant. Kyutai's Moshi is the one local speech-to-speech model out there, but it's still experimental — I wouldn't build on it yet.

Backend — the piece HA genuinely can't do:

- A lightweight FastAPI orchestration service holding session state (current step, active timers, substitutions) — exactly the stateful-conversation gap HA's trigger-action engine has
- Calls the LLM with recipe + conversation history + current step as context
- Talks to HA's API for device control and to Grocy/Mealie for inventory and recipes

- Streams audio to/from the tablet over websockets

Frontend:
Doesn't need to be native — a PWA or Flutter app in kiosk mode is usually less work than native Android/iOS, and still supports local wake-word listening (openWakeWord is a solid fully-local option) plus a clean step-by-step display.

One thing that changes the model picks a lot: what's the local server hardware — GPU or CPU-only?
yesterday

> **Claude Replied:**

What's the local server hardware for running the models?High-end GPU (16GB+ VRAM)

> **Claude Replied:**

Claude responded: With that much headroom, you can build a genuinely responsive setup:

Thought for 11s
Thought for 11s

With that much headroom, you can build a genuinely responsive setup:

LLM: Qwen2.5-14B-Instruct (Q4 or Q5 quant) is a strong pick — good tool-calling for recipe/HA actions, fits comfortably with room to spare. If you want more headroom for reasoning about substitutions or dietary constraints, Qwen2.5-32B at Q4 is also viable on 16GB+, just a bit slower per token. Serve it via Ollama for simplicity, or vLLM if you later want to handle multiple concurrent sessions efficiently.

STT + TTS fit alongside it easily: whisper-medium (~1.5GB) and XTTS (~3-4GB) can run concurrently with the LLM without fighting over VRAM — so you're not stuck with Piper's more robotic voice if you want XTTS's naturalness. Rough total: ~9GB for a Q4 14B model + ~5GB for STT/TTS, leaving slack.

Latency: this pushes round-trip down to roughly 1-1.5s, which is close enough to conversational that push-to-talk stops being necessary — worth designing for always-listening with a wake word from the start, since hands are usually messy mid-recipe. openWakeWord running locally on the tablet (cheap, doesn't touch the GPU) plus streaming STT on the server gets you a genuinely hands-free flow.

One architecture note: with this much compute, consider running STT/LLM/TTS as separate services behind the FastAPI orchestrator (rather than one monolith) — makes it easy to swap models later without touching session-state logic, and lets you scale each stage independently if you add more devices in the kitchen down the line.

yesterday

Claude Replied:

Claude responded: It doesn't interrupt the model — the model isn't running most of the time.

Thought for 7s
Thought for 7s

It doesn't interrupt the model — the model isn't running most of the time.

That's the thing that makes this simpler than it sounds. Between your turns, the orchestrator is parked on `await self.queue.get()`, doing nothing. When a timer fires, it puts an event on the queue and the orchestrator wakes up and processes it exactly like a user utterance. No interruption involved.

The timer task itself is trivial:

python

```python
async def run_timer(self, timer: Timer):
    delay = (timer.end_at - datetime.now()).total_seconds()
    await asyncio.sleep(delay)
    timer.status = "fired"
    await self.queue.put(TimerFired(timer))
```

Created with `asyncio.create_task` when `set_timer` is called, keeping the handle so `cancel_timer` can kill it.

The case you're actually asking about is when a timer fires while the model *is* generating. Then the queue holds the event until the current turn finishes and the loop comes back around to `queue.get()`. You wait out the rest of the response — typically a second or two — and then it speaks about the timer.

For cooking that's fine. Finishing "add the garlic and stir for about thirty seconds" before announcing the chicken is done is better behavior than cutting itself off mid-sentence.

If you did want preemption, you'd abort the in-flight generation the same way barge-in does — but I'd skip it. Nothing in a kitchen is urgent on a two-second timescale, and you'd be adding the trickiest failure mode in the system for no real gain.

One thing worth handling: **if several timers fire close together**, they queue as separate events and you'd get three consecutive announcements. Cheap fix is to drain the queue at the top of the loop and batch any timer events that arrived together into one synthesized turn, so the model can say "chicken's done and the rice is ready" in a single breath.

17 hours ago

You asked:

You said: Is the context reassmbled from scratch everytime and fed to the LLM as new or does the LLM retain context itself overtime

Is the context reassmbled from scratch everytime and fed to the LLM as new or
does the LLM retain context itself overtime

16 hours ago

Claude Replied:

Claude responded: From scratch, every time.

From scratch, every time. The model retains nothing.

Each `/api/chat` call is stateless — you send the complete message list, it generates a response, and it forgets everything. There's no session on Ollama's side holding your conversation. If you sent an empty message list on turn twenty, the model would have no idea it had ever spoken to you.

The illusion of memory comes entirely from you resending everything each turn.

What confuses this slightly: **the KV cache we tuned earlier is not memory.** It's a compute optimization — if the start of your prompt matches the previous call, llama.cpp skips recomputing those tokens. It makes calls faster; it doesn't preserve anything semantically. Clear it and behavior is identical, just slower.

That prefix-matching does have a practical implication for your design, though. Keeping stable content early in the prompt — system prompt, recipe, tool definitions — and volatile content late means more of the cache survives between turns. Put the timeline block (which changes every turn) at the top and you invalidate the cache on every call, forcing a full reprocess of everything. The block ordering in the spec was partly about this.

The upside of statelessness is that the model can't drift. It has no accumulated impressions, no half-remembered earlier version of the plan. Whatever your context assembler builds is exactly and only what it knows. Fix a bug in the timeline renderer and the next turn is immediately correct — there's nothing stale to flush.

This is also why the compression approach works at all. You're not "trimming the model's memory," you're just choosing what to include in a fresh document each turn. Dropping turn 4 from the transcript isn't forgetting; it's deciding that a rendered progress line represents it better.

16 hours ago